- **精灵图**
  - 使用精灵图核心：
    - 精灵技术主要针对于背景图片使用。就是把多个小背景图片整合到一张大图片中。
    - 这个大图片也称为 sprites 精灵图 或者 雪碧图
    - 移动背景图片位置， 此时可以使用 background-position 。
    - 移动的距离就是这个目标图片的 x 和 y 坐标。注意网页中的坐标有所不同
    - 因为一般情况下都是往上往左移动，所以数值是负值。
    - 使用精灵图的时候需要精确测量，每个小背景图片的大小和位置。
  - 使用精灵图核心总结：
    - 精灵图主要针对于小的背景图片使用。
    - 主要借助于背景位置来实现---background-position 。
    - 一般情况下精灵图都是负值。（千万注意网页中的坐标： x轴右边走是正值，左边走是负值， y轴同理）

- **字体图标**
  - 字体图标推荐下载网站：
    - 阿里 iconfont 字库 http://www.iconfont.cn/
    - icomoon 字库 http://icomoon.io
  - 字体图标的引入
    - 把下载包里面的五个不认识格式的文件放入页面根目录下的新建文件夹fonts中。（html文件是说明文档）

名称	修改日期	类型	大小
demo.css	2021/4/21 11:18	层叠样式表文档	9 KB
demo_index.html	2021/4/21 11:18	Chrome HTML D...	10 KB
iconfont.css	2021/4/21 11:18	层叠样式表文档	3 KB
iconfont.eot	2021/4/21 11:18	EOT 文件	3 KB
iconfont.js	2021/4/21 11:18	JavaScript 文件	6 KB
iconfont.json	2021/4/21 11:18	JSON 文件	1 KB
iconfont.svg	2021/4/21 11:18	Microsoft Edge ...	5 KB
iconfont.ttf	2021/4/21 11:18	TrueType 字体文件	3 KB
iconfont.woff	2021/4/21 11:18	WOFF 文件	2 KB
iconfont.woff2	2021/4/21 11:18	WOFF2 文件	2 KB

不同浏览器所支持的字体格式是不一样的，字体图标之所以兼容，就是因为包含了主流浏览器支持的字体文件。

1. TureType(.ttf)格式.ttf字体是Windows和Mac的最常见的字体，支持这种字体的浏览器有IE9+、Firefox3.5+、Chrome4+、Safari3+、Opera10+、iOS Mobile、Safari4.2+；
2. Web Open Font Format(.woff)格式woff字体，支持这种字体的浏览器有IE9+、Firefox3.5+、Chrome6+、Safari3.6+、Opera11.1+；
3. Embedded Open Type(.eot)格式.eot字体是IE专用字体，支持这种字体的浏览器有IE4+；
4. SVG(.svg)格式.svg字体是基于SVG字体渲染的一种格式，支持这种字体的浏览器有Chrome4+、Safari3.1+、Opera10.0+、iOS Mobile Safari3.2+；

- 在 CSS 样式中全局声明字体： 简单理解把这些字体文件通过css引入到我们页面中（一定注意字体文件路径的问题）。

```
/* 改路径之前 */
@font-face {
    font-family: 'iconfont';
    src: url('iconfont.eot');
    src: url('iconfont.eot?#iefix') format('embedded-opentype'),
    url('iconfont.woff2') format('woff2'),
    url('iconfont.woff') format('woff'),
    url('iconfont.ttf') format('truetype'),
    url('iconfont.svg#iconfont') format('svg');
}

/* 改路径之后（字体文件夹名字为fonts且在根目录） */
@font-face {
    font-family: 'iconfont';
    src: url('./fonts/iconfont.eot');
    src: url('./fonts/iconfont.eot?#iefix') format('embedded-opentype'),
    url('./fonts/iconfont.woff2') format('woff2'),
    url('./fonts/iconfont.woff') format('woff'),
    url('./fonts/iconfont.ttf') format('truetype'),
    url('./fonts/iconfont.svg#iconfont') format('svg');
}
```

- html 标签内添加小图标。
  - 在正常标签中添加

```
<span class="iconfont">&#xe603;</span>
```

- 在伪元素中添加

```
span::after {
    content: "\e603";/* 复制后四位，且前面加个\ */
}
```

- 给标签定义字体。
  - 正常标签中的字体图标。

```
<style>
    .iconfont {
        font-family: "iconfont" !important;
        font-size: 16px;
        font-style: normal;
        -webkit-font-smoothing: antialiased;
        -moz-osx-font-smoothing: grayscale;
    }
</style>

<body>
    <span class="iconfont">&#xe603;</span>
</body>
```

- 伪元素中的字体图标。

```
<style>
    span::after {
        content: "\e603";
        font-size: 16px;
        font-style: normal;
        -webkit-font-smoothing: antialiased;
        -moz-osx-font-smoothing: grayscale;
    }
</style>
```

        - 注意：
            - 这种方法如果图标和文字对不齐，可加一条vertical-align: middle;进行调节
            - 如果还是对不齐，可以再调节字体图标的行高（需要先把这个伪元素设置成行内块或者块并使其高度高于字体图标）
    - 字体图标的追加
        - 如果工作中，原来的字体图标不够用了，我们需要添加新的字体图标到原来的字体文件中。
        - 重新下载文件，把压缩包里面的不认识的五个文件替换一下。

- **CSS 三角**
    - 一张图， 你就知道 CSS 三角是怎么来的了, 做法如下：

    - 代码：

```
div {
    width: 0;
    height: 0;
    line-height: 0;
    font-size: 0;
    border: 50px solid transparent;
    border-left-color: pink;
}
```

- **CSS 用户界面样式**

- 所谓的界面样式，就是更改一些用户操作样式，以便提高更好的用户体验。
  - 更改用户的鼠标样式
  - 表单轮廓
  - 防止表单域拖拽
- 鼠标样式 cursor
  - 代码：

```
li {
    cursor: pointer;
}
```

  - 设置或检索在对象上移动的鼠标指针采用何种系统预定义的光标形状。

属性值	描述
default	小白 默认
pointer	小手
move	移动
text	文本
not-allowed	禁止

- 轮廓线 outline
  - 给表单添加 outline: 0; 或者 outline: none; 样式之后，就可以去掉默认的蓝色边框。

```
input {
    outline: none;
}
```

- 防止拖拽文本域 resize
  - 实际开发中，我们文本域右下角是不可以拖拽的。

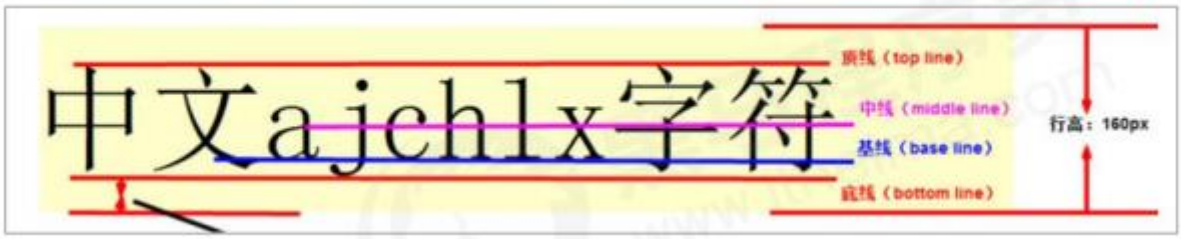
```
textarea{
    resize: none;
}
```

- **vertical-align 属性介绍**

- CSS 的 vertical-align 属性使用场景： 经常用于设置图片或者表单(行内块元素）和文字垂直对齐。
- 官方解释： 用于设置一个元素的垂直对齐方式，但是它只针对于行内元素或者行内块元素有效。
- 语法：

```
vertical-align : baseline | top | middle | bottom
```

值	描述
baseline	默认。元素放置在父元素的基线上。
top	把元素的顶端与行中最高元素的顶端对齐
middle	把此元素放置在父元素的中部。
bottom	把元素的顶端与行中最低的元素的顶端对齐。



- **vertical-align 属性应用**
  - 图片、表单和文字对齐
    - 图片、表单都属于行内块元素，默认的 vertical-align 是基线对齐。

    - 此时可以给图片、表单这些行内块元素的 vertical-align 属性设置为 middle 就可以让文字和图片垂直 居中对齐了。
  - 解决图片、iframe底部默认空白缝隙问题
    - bug：图片、iframe底侧会有一个空白缝隙，原因是行内块元素会和文字的基线对齐。
    - 主要解决方法有两种：
      - 添加 vertical-align:middle | top| bottom 等。 （提倡使用的）
      - 转换为块级元素 display: block;

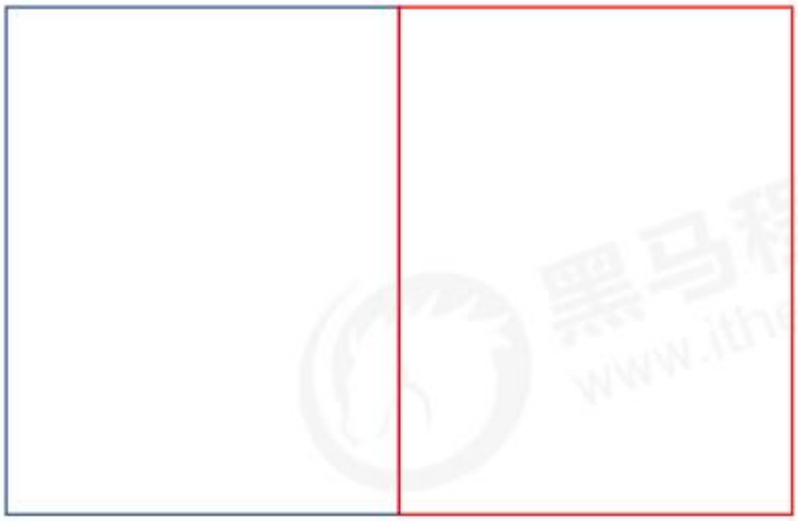
- **溢出的文字省略号显示**
  - 单行文本溢出显示省略号--必须满足三个条件

```
/*强制一行内显示文本*/
white-space: nowrap;  ( 默认 normal 自动换行)
/*文字用省略号替代超出的部分*/
text-overflow: ellipsis;
/*超出的部分隐藏*/
overflow: hidden;
```

  - 多行文本溢出显示省略号
    - 多行文本溢出显示省略号，有较大兼容性问题， 适合于webKit浏览器或移动端（移动端大部分是webkit内核）

```
/* 弹性伸缩盒子模型显示 */
display: -webkit-box;
/* 设置或检索伸缩盒对象的子元素的排列方式 */
-webkit-box-orient: vertical;
/* 限制在一个块元素显示的文本的行数 */
-webkit-line-clamp: 2;
/*超出的部分隐藏*/
overflow: hidden;
```

- **常见布局技巧**
  - margin负值运用



- 让每个盒子margin 往左侧移动 -1px 正好压住相邻盒子边框
- 鼠标经过某个盒子的时候，提高当前盒子的层级即可（如果没有有定位，则加相对定位（保留位置），如果有定位，则加z-index）

- 文字围绕浮动元素

- 巧妙运用浮动元素不会压住文字的 特性

- 行内块巧妙运用（页码在页面中间显示:）

- 把这些链接盒子转换为行内块， 之后给父级指定 text-align:center;
- 利用行内块元素中间有缝隙，并且给父级添加 text-align:center; 行内块元素会水平会居中

- CSS 三角强化

- 原理：

- 代码：

```
width: 0;
height: 0;
border-color: transparent red transparent transparent;
border-style: solid;
border-width: 22px 8px 0 0;
```

- CSS 初始化
  - 不同浏览器对有些标签的默认值是不同的，为了消除不同浏览器对HTML文本呈现的差异，照顾浏览器的兼容，我们需要对CSS 初始化。
  - 简单理解： CSS初始化是指重设浏览器的样式。（也称为CSS reset）。每个网页都必须首先进行 CSS初始化。这里我们以 京东CSS初始化代码为例。
  - Unicode编码字体：把中文字体的名称用相应的Unicode编码来代替，这样就可以有效的避免浏览器解释CSS代码时候出现乱码的问题。
  - 比如：黑体 \9ED1\4F53、宋体 \5B8B\4F53、微软雅黑 \5FAE\8F6F\96C5\9ED1。